Algorithms Optimization

José Anastacio Hernández Saldaña M.C.

September 30, 2026

Part 1: Algorithms that Use Multiple States

I. Algorithm Optimization

"Optimizing algorithms is not just about writing faster code; it is about changing how state and time are explored."

Optimization Foundations

- **Time Complexity:** Moving from deterministic single-path execution to concurrent search spaces.
- **State Traversal:** Exploring many states (branches, sub-problems, samples) at the same time instead of one after another.
- **Architectural Shift:** Adapting algorithmic logic to multi-core (Parallel) and networked cluster (Distributed) topologies.

II. Multi-State Algorithms & Parallel Execution

An algorithm that keeps several **independent** states alive at the same time can process them on several cores at the same time.

Single-State vs. Multi-State Search

- **Single-State:** Keeps one current state; each step depends on the previous one, so the steps cannot run at the same time.
  - **Examples:** Binary search (one interval), Newton's method ($x_{k+1} = x_k - f(x_k)/f'(x_k)$), Euclid's GCD.
- **Multi-State:** Keeps many states that do not depend on each other, so they can be processed in parallel.

- **Examples:** BFS (the whole frontier of a level), Merge sort (independent halves), Monte Carlo simulation (independent random samples).

Parallel vs. Looping: Independent Runs

Many algorithms repeat the same computation with different inputs or random seeds (Monte Carlo simulation, parameter sweeps, cross-validation folds):

- **Looping (Sequential Runs):** Run $k+1$ waits for run k to finish. Idle CPU cores, high wall-clock latency.
- **Parallel Runs:** M runs start on M worker processes at the same time, each with its own input or random seed.
  - **Embarrassingly Parallel:** Near-zero inter-process communication required during exploration.
  - **Early Termination:** In a search (e.g., the first divisor of a large number), the first worker that finds the answer signals the others to stop.

Parallel Implementation in Python

In Python, CPU-bound tasks require `multiprocessing` rather than `threading` due to the **Global Interpreter Lock (GIL)**.

```
import multiprocessing as mp
import random
import time

def estimate_pi(seed: int, samples: int = 1_000_000)
    -> float:
    """Monte Carlo: fraction of random points that
       fall inside the unit quarter circle."""
    rng = random.Random(seed)
    inside = sum(rng.random()**2 + rng.random()**2 <=
        1.0 for _ in range(samples))
    return 4 * inside / samples

if __name__ == "__main__":
```

```
    seeds = list(range(8))  # 8 independent runs, one
        seed each

    # 1. Sequential Looping
    t0 = time.perf_counter()
    seq_results = [estimate_pi(s) for s in seeds]
    t_seq = time.perf_counter() - t0

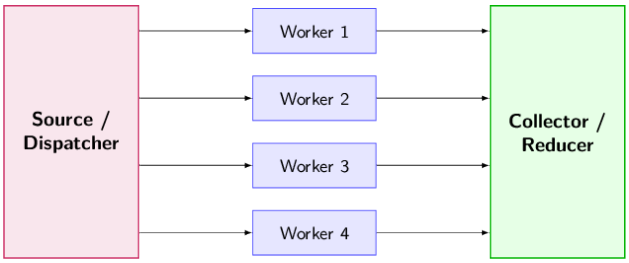
    # 2. Parallel Pool Execution
    t0 = time.perf_counter()
    with mp.Pool(processes=mp.cpu_count()) as pool:
        par_results = pool.map(estimate_pi, seeds)
    t_par = time.perf_counter() - t0

    print(f"Sequential wall-clock: {t_seq:.3f} s")
    print(f"Parallel wall-clock:   {t_par:.3f} s (
        Speedup: {t_seq/t_par:.2f}x)")
    print(f"pi estimate (mean of {len(seeds)} runs): {
        sum(par_results) / len(par_results):.5f}")
```

```
Sequential wall-clock: 1.630 s
Parallel wall-clock:   0.326 s (Speedup: 4.99x)
pi estimate (mean of 8 runs): 3.14126
```

The Collector Pattern

`pool.map` above is this pattern: the pool dispatches seeds to workers, and the mean reduces (collects) their estimates.



III. Distributed Algorithms & System Theory

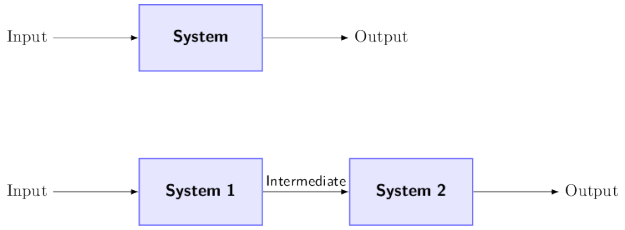
Scaling beyond single-node shared memory requires distributed computation and formal system modeling.

Systems Theory in Pipeline Engineering

A system is a purposeful collection of interconnected components with defined inputs, transformations, and outputs.

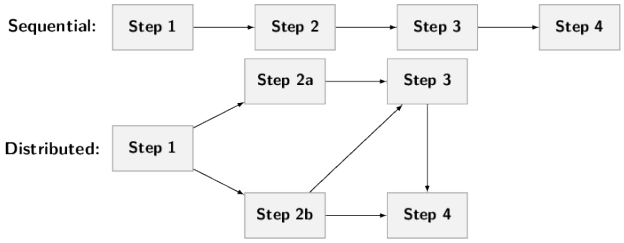
- **Key Concepts:**
  - **Synergy:** Combined subsystem output exceeds individual totals.
  - **Emergence:** System-level behaviors not present in isolated subcomponents.
- **Sub-Disciplines:** System dynamics, reliability engineering, mathematical optimization, and operational modeling.

System Theory: Basic and Composed Models



Execution Topologies Compared

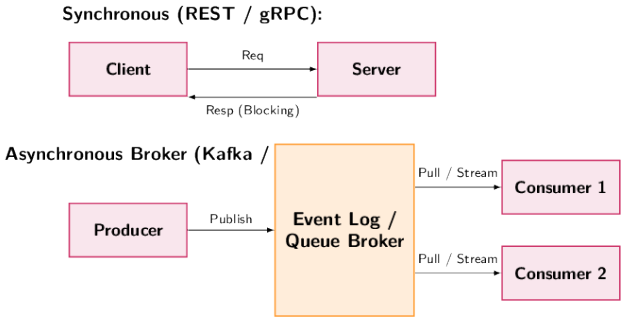
Transforming algorithms from sequential loops to distributed task and data execution graphs.



IV. Distributed Communication: Synchronous APIs vs. Message Brokers

Distributed algorithms communicate across network boundaries either via point-to-point RPC/REST calls or decoupled message brokers.

Point-to-Point APIs vs. Broker Architecture



Architectural Comparison

Dimension	Synchronous APIs (REST / gRPC)	Message Brokers (Kafka / RabbitMQ)
Coupling	High temporal coupling (both online)	Loose temporal decoupling
Flow Control	Hard backpressure; caller can fail	Natural buffer (log offsets / queue)
Topology	Point-to-point (1:1)	Publish-Subscribe (1:N) / Worker pool
State	Stateless endpoints; remote database	Persistent, distributed commit log
Failure Mode	Cascading timeouts under heavy load	Safe message queuing during outages

V. Functional Programming: The Concurrency Engine

Distributed computing is fundamentally constrained by mutable state. Functional Programming (FP) eliminates shared mutable memory.

The Three Pillars of FP for Systems

- **Pure Functions:** $f(x)$ depends solely on argument x. Referential transparency ensures mathematical repeatability.
- **Immutability:** Data structures are never mutated in place. State transformations yield fresh values, eliminating race conditions and lock contention.
- **First-Class Functions:** Functions are values passed into higher-order combinators (`map`, `filter`, `fold/reduce`).

Pure Function vs. Destructive Mutation

```
# Destructive State Mutation (Unsafe for Concurrency)
global_accumulator = 0
def impure_sink(val: int) -> int:
    global global_accumulator
    global_accumulator += val  # Shared mutable state:
        Race condition under threads
    return global_accumulator

# Pure Mathematical Function (Thread-Safe &
    Distributable)
def pure_transform(val: int) -> int:
    return val * 2  # No side effects, deterministic
        output

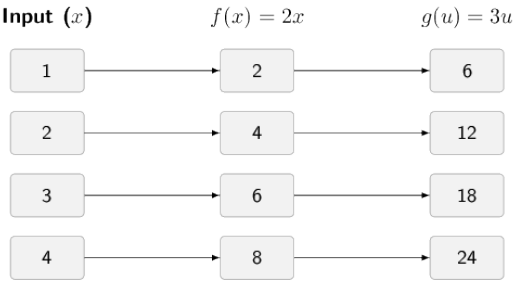
data = [1, 2, 3, 4, 5]
transformed = list(map(pure_transform, data))
print(f"Transformed stream: {transformed}")
```

```
Transformed stream: [2, 4, 6, 8, 10]
```

Mathematical Function Composition

In functional pipelines, multi-stage transformations mirror continuous mappings:

$$(g \circ f)(x) = g(f(x))$$



VI. Reactive Programming & Event Streams

Functional Reactive Programming (FRP) combines functional operations with asynchronous event streams.

The Reactive Paradigm

- **Stream as a First-Class Citizen:** An unbounded sequence of events ordered in time: $\mathcal{S} = [e_0, e_1, e_2, \dots]$.
- **Push vs. Pull:**
  - **Iterators (Pull):** Consumer requests data when ready: $T =$ next().
  - **Observables (Push):** Producer pushes data onto the pipeline as events arrive: onNext(T) $\rightarrow$ onError() | onComplete().
- **Backpressure:** The mechanism allowing a downstream consumer to signal an upstream producer to slow down or buffer emission rates.

Reactive Pipeline Example: Generators & Operators

```
from typing import Iterator, Callable

def event_source() -> Iterator[int]:
    """Simulates an asynchronous stream of arriving
       events."""
    for item in range(1, 10):
        yield item

def stream_filter(predicate: Callable[[int], bool],
   stream: Iterator[int]) -> Iterator[int]:
    for event in stream:
        if predicate(event):
            yield event

def stream_map(transform: Callable[[int], int], stream
   : Iterator[int]) -> Iterator[int]:
    for event in stream:
        yield transform(event)

# Constructing the reactive stream
raw_stream = event_source()
evens = stream_filter(lambda x: x % 2 == 0, raw_stream
   )
scaled = stream_map(lambda x: x * 10, evens)
```

```
print("Reactive output events:", list(scaled))
```

```
Reactive output events: [20, 40, 60, 80]
```

VII. Key Takeaways: Transforming Algorithms

- **Decoupling State from Execution:** Replace shared memory with immutable, serialized message payloads.
- **Partitioning Strategy:** Split computational workloads via data partitioning (sharding) or task graphs (DAGs).
- **Failure Tolerance:** Distributed nodes require idempotent functions and deterministic retries to handle partial network or hardware outages.

Part 2: Algorithms that Use Problem Information

I. Motivation: Problem Information and Complexity

The Challenge of NP-Hard Problems

- **Computational Intractability:** Many real-world optimization problems (TSP, Knapsack, Scheduling) are NP-hard.
- $\mathcal{P}$ **vs** $\mathcal{NP}$**:** No known deterministic algorithm can solve them in polynomial time $\mathcal{O}(n^k)$.
- **Exact vs. Approximate:** Do we demand the absolute best solution in centuries (Exact algorithms like Branch & Bound), or a "good enough" one in seconds?

Exact Algorithms: Deterministic, Sound and Complete

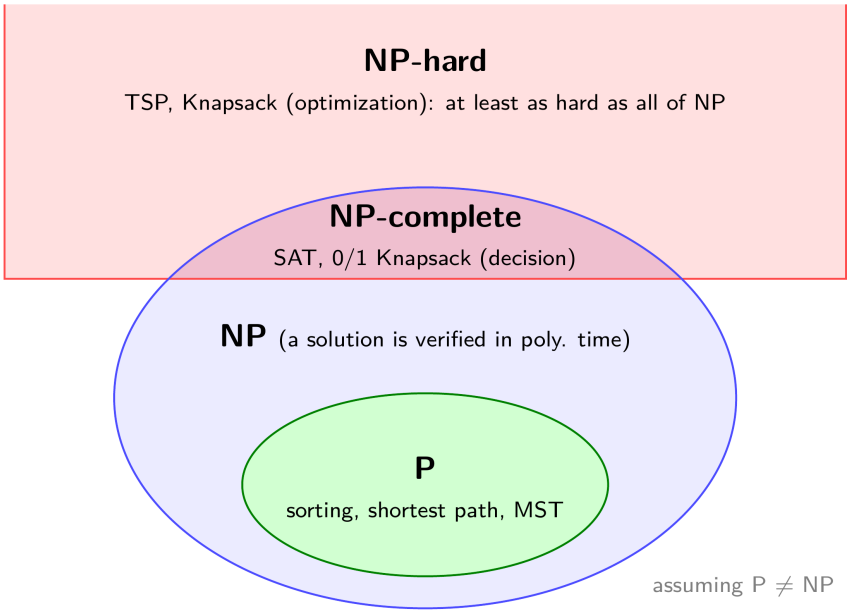
An **exact** algorithm always returns a proven optimum. It is usually **deterministic**: the same input always follows the same steps and gives the same output.

- **Sound:** Every answer it returns is correct: the solution is feasible and truly optimal.

- **Complete:** If a solution exists, it finds it (and proves optimality) in finite time.
- **The price:** For NP-hard problems, every known exact algorithm needs exponential time in the worst case ($\mathcal{O}(2^n)$, $\mathcal{O}(n!)$).
- **Examples:** Brute force, Branch & Bound, Dynamic Programming, ILP solvers.

P, NP and NP-hard

- $\mathcal{P}$: solvable in polynomial time by a **deterministic** algorithm.
- $\mathcal{NP}$: a proposed solution can be **verified** in polynomial time. Equivalently, a **nondeterministic** machine that always "guesses" the right choice solves it in polynomial time.
- NP-complete $= NP \cap NP$-hard: the hardest problems inside $\mathcal{NP}$.



- **Nondeterministic** is a theoretical "perfect guesser", **not** a random algorithm: no real machine guesses right every time.
- A deterministic exact algorithm for an NP-hard problem runs in exponential time (unless $\mathcal{P} = \mathcal{NP}$).

- Randomness does not make NP-hard problems polynomial. What it buys is **speed in exchange for guarantees**.

Stochastic Algorithms: Trading Guarantees for Time

A **stochastic** (randomized) algorithm makes random choices: the same input can produce different runs and different outputs.

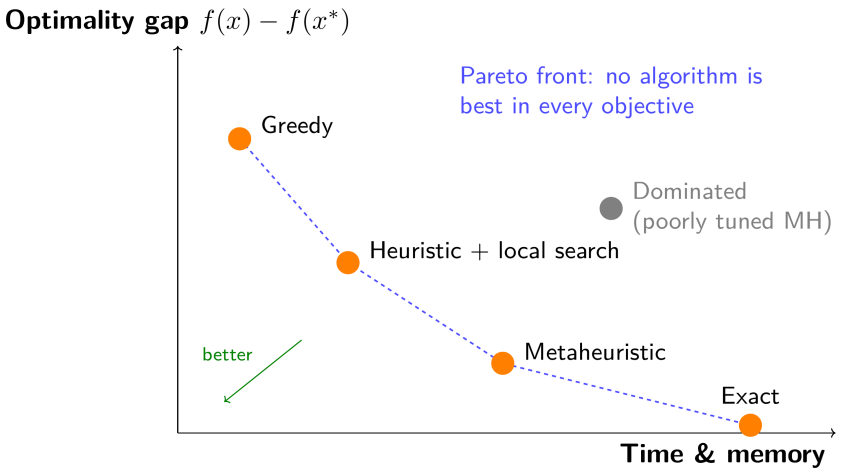
- **Two families: Las Vegas** (always correct, random running time) and **Monte Carlo** (fixed running time, probably correct), studied in Part 3.
- **Stochastic search for NP-hard optimization:** Each run takes polynomial time, but it gives up soundness and completeness: it returns a **good** solution, not a **proven optimal** one.
- **Repetition helps:** More runs or restarts raise the probability of reaching the optimum, but never turn it into a guarantee.

The Information Bias

- **Quality vs. Resources:** Once a solution is **good enough**, the question changes: how much CPU time and memory are we willing to spend for each extra step towards the true optimum $f(x^*)$?
- **Intelligent Choice:** A **Heuristic** is problem-specific **information** used to "intelligently choose" or bias decisions, expecting to yield a better solution faster than blind search.
- **The Rule:** The more problem-specific information we embed into the search logic, the faster we can find high-quality approximations.

Quality vs. Resources: A Multi-objective Trade-off

Choosing an algorithm is itself a **multi-objective problem**: minimize time, minimize memory, and minimize the optimality gap at the same time.



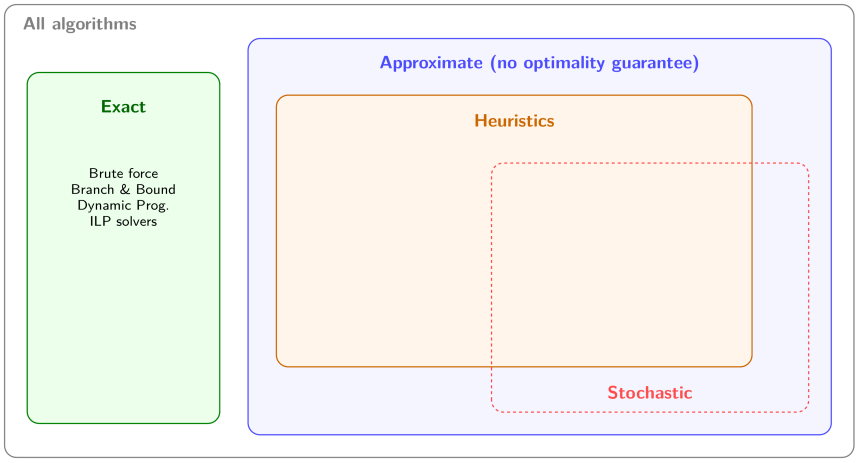
- An algorithm is **dominated** when another one is at least as good in every objective and better in one.
- The rest of this part moves along this front: from cheap greedy choices to expensive exact methods.

II. Heuristics: Exploiting Domain Information

What is a Heuristic?

A heuristic is a problem-specific rule of thumb: it biases the exploration of the search space with domain information, ignoring regions based on human-engineered rules.

- Heuristics are **approximate** algorithms: fast, but with no guarantee of optimality (not sound, not complete).
- Many are **stochastic** (random starts, random tie-breaking, random neighbours); others are deterministic (Nearest Neighbour, 2-opt).

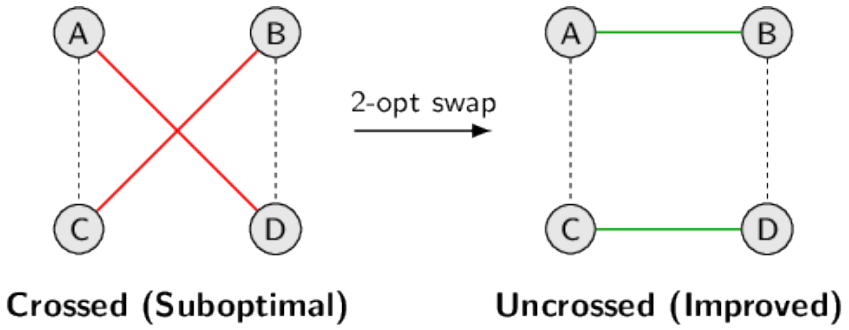


Constructive vs. Improvement Heuristics

- **Constructive Heuristics:** Build a solution iteratively from an empty state.
  - **Examples:** Nearest Neighbour (TSP), First-Fit Decreasing (Bin Packing), Christofides algorithm.
- **Improvement Heuristics (Local Search):** Take a complete solution and apply localized perturbations to find a better neighbor.
  - **Examples:** 2-opt / 3-opt (Routing), Kernighan-Lin (Graph Partitioning), Bit-flip / Swap (Scheduling).
- **The Limitation:** They are deterministic (or use naive randomness) and strictly hill-climbing. They **will** get stuck in local optima.

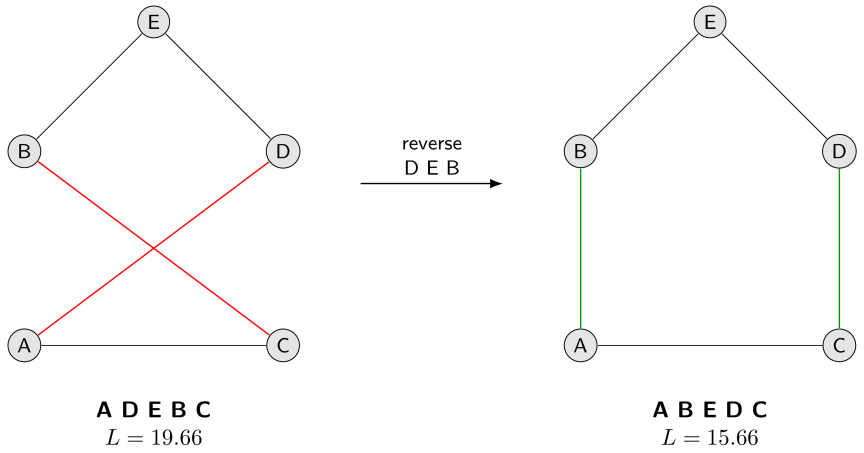
Example: 2-Opt Local Search (TSP)

Improvement heuristics scan the neighborhood for better states. In TSP, 2-opt removes two crossing edges and reconnects them to shorten the tour.



2-Opt Step by Step: A 5-City Route

Cities: $A(0,0)$, $B(0,3)$, $E(2,5)$, $D(4,3)$, $C(4,0)$. The tour $A \to D \to E \to B \to C \to A$ has two crossing edges, (A, D) and (B, C).



2-opt removes the two crossing edges and reconnects the tour by **reversing the segment** between them:

```
import math

cities = {"A": (0, 0), "B": (0, 3), "E": (2, 5), "D":
    (4, 3), "C": (4, 0)}

def tour_length(route: list[str]) -> float:
    return sum(math.dist(cities[route[i]], cities[
        route[(i + 1) % len(route)]])
                for i in range(len(route)))
```

```
def two_opt_move(route: list[str], i: int, k: int) ->
    list[str]:
     """Reverse route[i..k]: removes edges (i-1, i) and
         (k, k+1)."""
     return route[:i] + route[i:k + 1][::-1] + route[k
        + 1:]

route = ["A", "D", "E", "B", "C"]
new_route = two_opt_move(route, 1, 3)
print(f"Before: {route}  L = {tour_length(route):.2f}"
    )
print(f"After:  {new_route}  L = {tour_length(
    new_route):.2f}")
```

```
Before: ['A', 'D', 'E', 'B', 'C']  L = 19.66
After:  ['A', 'B', 'E', 'D', 'C']  L = 15.66
```

- A full 2-opt local search repeats this move for every pair (i, k) until no reversal shortens the tour.

Local Search and Optimization Problems

- **Local Search**: Algorithms that operate using a single current state (or small set) and move only to neighbors.
- **Advantages**:
  1. Uses very little memory (constant space).
  2. Can find reasonable solutions in large/infinite state spaces where systematic search fails.
- **Optimization Problems**: Aim to find the best state according to an **objective function**.

III. Greedy Algorithms: The Short-Sighted Guide

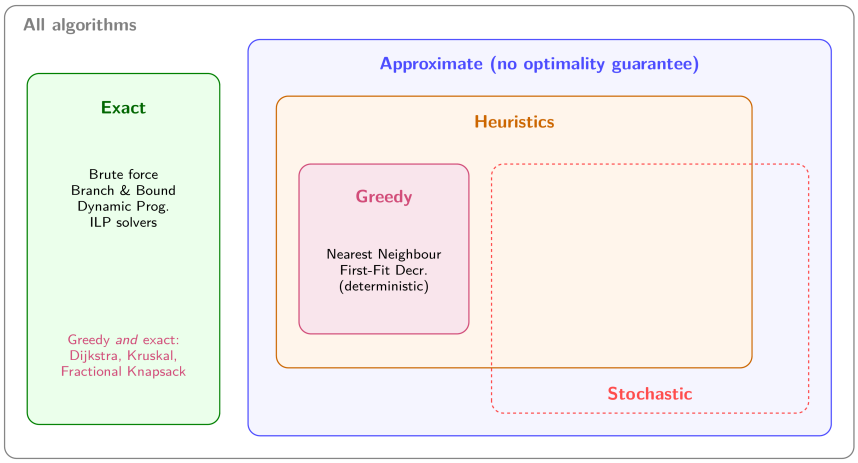
Principles of Greedy Choice

- **Principle of Optimality (Bellman):** An optimal solution is made of optimal solutions to its subproblems: whatever the first decision, the remaining decisions must be optimal for the state it leaves.

- **Greedy algorithms stick to it at every step:** They always take the locally optimal decision and never allow a non-optimal one, and they never undo a decision.
- **Greedy-Choice Property:** Assumes a global optimum can be reached by making a locally optimal choice at each step.
- **The Flaw (Short-Sightedness):** Greedy algorithms lack vision. They lock in decisions based purely on immediate, localized payoff without considering the holistic impact.
- **When they succeed:** Problems with optimal substructure (e.g., Fractional Knapsack, Dijkstra's shortest path).

Where Greedy Fits

Greedy is a family of heuristics that follows the principle of optimality blindly. When the greedy choice is provably optimal, it is also an exact algorithm.



The 0/1 Knapsack Problem

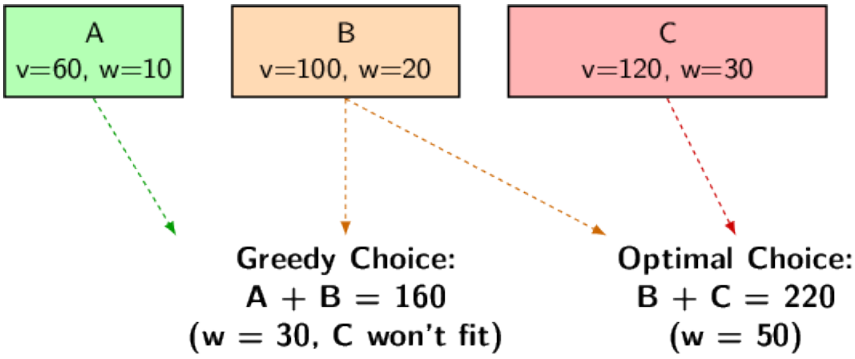
Given n items, each with value v_i and weight w_i, and a knapsack of capacity W, choose the items that maximize the total value without exceeding the capacity.

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^{n} w_i x_i \leq W, \qquad x_i \in \{0,1\}$$

- $x_i = 1$ if item i goes into the knapsack, 0 otherwise.
- **Fractional Knapsack** relaxes $x_i \in [0, 1]$: there, greedy by v_i/w_i is optimal.
- **0/1 Knapsack** is \$NP\$-hard: greedy by v_i/w_i can fail.

Why Greedy Fails: 0/1 Knapsack Example

- **Capacity:** $W = 50$
- **Items:** $A(v = 60, w = 10)$, $B(v = 100, w = 20)$, $C(v = 120, w = 30)$
- **Greedy Logic:** Sort by ratio (v/w). $A = 6, B = 5, C = 4$.



The Secretary Problem

- n candidates are interviewed one at a time, in random order.
- After each interview we must **hire now or reject forever**; a rejected candidate never comes back.
- **Goal:** maximize the probability of hiring **the best** candidate.
- **Greedy rule:** Hire as soon as the current candidate is the best seen so far. The first candidate always is, so greedy always hires the first one: $P(\text{best}) = 1/n \to 0$.
- **Optimal stopping ($1/e$ rule):** Reject the first $r \approx n/e$ ($\approx 37\%$) candidates, then hire the first one better than all of them: $P(\text{best}) \to 1/e \approx 0.37$.
- **Lesson:** Sacrificing immediate payoff to **explore** first pays off. Metaheuristics use the same idea when they accept bad moves.

Greedy vs. Optimal Stopping (Simulation)

```
import math
import random

def hire(candidates: list[int], r: int) -> int:
    """Observe the first r candidates, then hire the
       first better than all of them."""
    best_seen = max(candidates[:r], default=-1)
    return next((c for c in candidates[r:] if c >
       best_seen), candidates[-1])

n, trials = 100, 20_000
rules = {"Greedy (r = 0)": 0, "1/e rule (r = 37)":
   round(n / math.e)}
for name, r in rules.items():
    wins = sum(hire(random.sample(range(n), n), r) ==
       n - 1 for _ in range(trials))
    print(f"{name:18s} P(best) = {wins / trials:.3f}")
```

```
Greedy (r = 0)     P(best) = 0.011
1/e rule (r = 37)  P(best) = 0.373
```

- **Similar problems:** selling a house or an apartment hunt (accept or reject offers), parking (take a spot or drive on), online ad allocation and online matching, the prophet inequality, and the exploreexploit dilemma of multi-armed bandits.

Hill-climbing Search

Known as "Greedy local search".

- **Algorithm**: Loop that continually moves in the direction of increasing value (steepest ascent). Terminates when no neighbor is higher.

```
function HILL–CLIMBING(problem) returns a
   state
  current = problem.INITIAL_STATE
  loop do
    neighbor = HIGHEST_VALUED_SUCCESSOR(
       current)
```

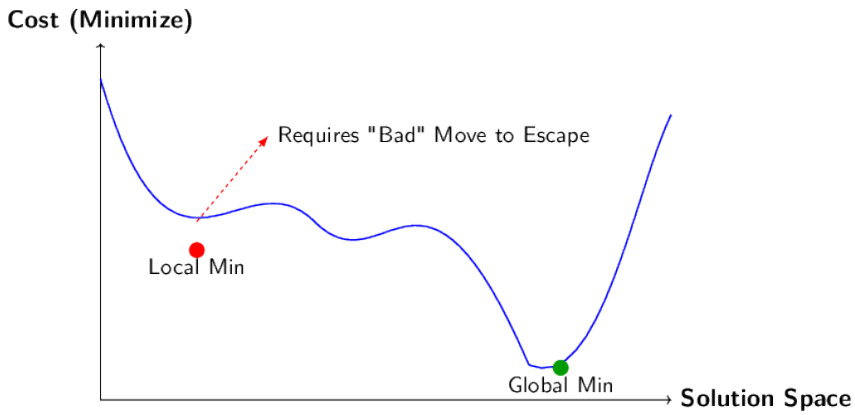
```
    if neighbor.VALUE <= current.VALUE then
      return current
    current = neighbor
```

- **Metaphor**: Climbing Everest in thick fog with amnesia.
- **Drawbacks** (Getting stuck):
  1. **Local Maxima**: A peak higher than neighbors but lower than the global maximum.
  2. **Ridges**: Sequence of local maxima; difficult to navigate (requires diagonal moves).
  3. **Plateaus**: Flat area. Can be a flat local maximum (stuck) or a shoulder (progress possible).
- **Variants**:
  - **Stochastic Hill Climbing**: Choose at random from uphill moves (slower but more complete).
  - **First-choice Hill Climbing**: Generate successors randomly until one is better than current.
  - **Random-restart Hill Climbing**: Runs search repeatedly from random starts. Complete with probability ≈ 1 (each restart is an independent run: see *Parallel vs. Looping* in Part 1).

The Optimization Landscape Trap

Greedy and basic local search algorithms inevitably get trapped in local minima because they refuse to accept worse solutions.

- **Elevation** = objective value or cost: Hill Climbing looks for the **global maximum**, Gradient Descent for the **global minimum** (plotted here).



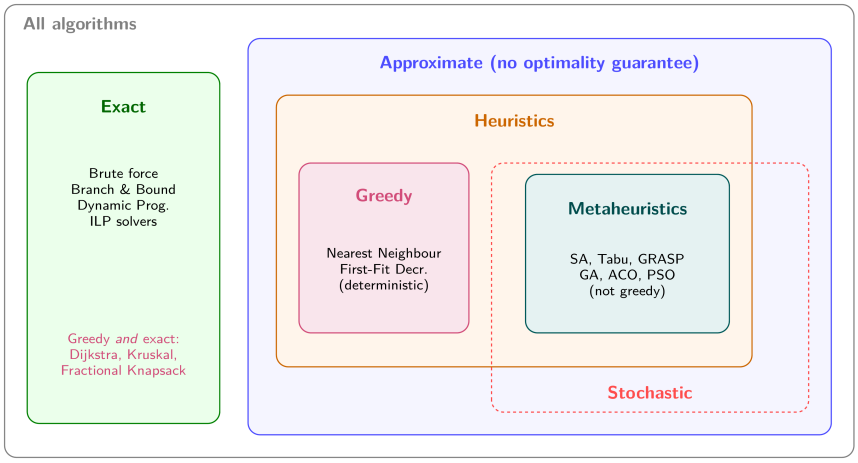
IV. Metaheuristics: Escaping the Local Minima

Metaheuristics: Escaping the Trap

- **Definition:** Problem-independent, high-level frameworks that guide and manage lower-level heuristics. They are the "escape hatch" from local optima.
- **The Core Philosophy:** To reach the global minimum, a metaheuristic includes criteria to accept **bad** decisions temporarily.
- **Not Greedy:** They deliberately take non-optimal decisions (worse moves, random choices), breaking the greedy commitment to the principle of optimality.
- **Intensification (Exploitation):** Searching thoroughly around currently good solutions.
- **Diversification (Exploration):** Exploring unvisited regions of the search space.

Where Metaheuristics Fit

Metaheuristics are heuristics too, but they are not greedy, and most of them are stochastic.



Taxonomy of Metaheuristics

The single-state / multi-state split from Part 1 appears again: population-based methods keep many independent states, so they parallelise naturally.

Category	Characteristics
Single-Solution (Trajectory)	Traces one path through the landscape, accepting bad moves to es
Population-Based	Evolves a set of solutions that share information about good regio

Simulated Annealing

- Combines **hill climbing** (efficiency) with **random walk** (completeness).
- **Analogy**: Metallurgy (heating metal and cooling slowly to reach low-energy crystalline state).
  - **Mechanism**:
    - Pick a random move.
    - If it improves the situation ($ Δ E $>$ 0 $): **Always accept**.
    - If it worsens the situation ($ Δ E $<$ 0 $): Accept with probability $ P $= e^{$\Delta$ E / T} $.
  - **Temperature (T)**: Starts high (shakes hard, accepts bad moves to escape local maxima) and decreases to 0 (pure hill climbing).
- **Algorithm**:

```
function SIMULATED-ANNEALING(problem,
    schedule) returns a state
  current = problem.INITIAL_STATE
  for t = 1 to infinity do
    T = schedule(t)
    if T = 0 then return current
    next = RANDOM_SUCCESSOR(current)
              # Diff 1: Random pick (not
       best)
    delta_E = next.VALUE - current.VALUE
    if delta_E > 0 then
      current = next
    else
      current = next with probability e
         ^(delta_E / T) # Diff 2: Accept
          bad moves
```

Local Beam Search (Multi-start)

- Keeps track of k states instead of just one.
- **Algorithm**:
  1. Generate all successors of all k states.
  2. If any is a goal, halt.
  3. Else, select the k best successors from the complete list and repeat.
- **Key Difference vs Random Restarts**: Threads communicate. Information is passed; if one finds a good area, others move there.
- **Problem**: Lack of diversity (states cluster together).
- **Fix**: **Stochastic Beam Search** (choose successors proportional to value, not just top k).

GRASP (Greedy Randomized Adaptive Search Procedure)

- A multi-start metaheuristic that repeatedly builds valid solutions and then improves them.
- **Structure**: Each iteration has two distinct phases:

1. **Construction Phase**: Build a feasible solution one element at a time (e.g., adding cities to a tour).
   - **Greedy**: List best candidates.
   - **Randomized**: Don't pick the #1 best. Pick randomly from the top candidates (Restricted Candidate List - RCL).
   - **Adaptive**: The "greediness" (cost) is re-evaluated after every step.
2. **Local Search Phase**: Apply Hill Climbing (or similar) to the constructed solution to reach a local optimum.

- **Algorithm**:

```
function GRASP(max_iterations) returns
    best_solution
  best_solution = infinity
  loop max_iterations times do
    ; Phase 1: Build a rough solution
    candidate =
       CONSTRUCT_GREEDY_RANDOMIZED_SOLUTION
       ()

    ; Phase 2: Refine it to a local optimum
    candidate = LOCAL_SEARCH(candidate)

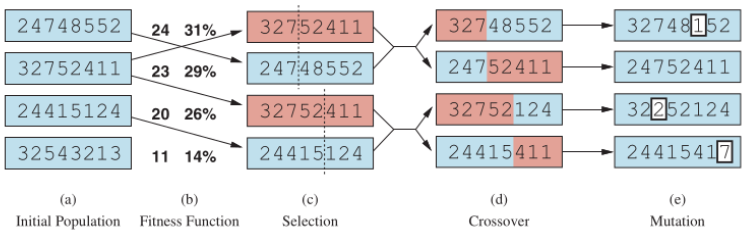
    if candidate.VALUE > best_solution.VALUE
        then
      best_solution = candidate
  return best_solution
```

  - **Key Parameters**:
    - **RCL (Restricted Candidate List)**: Controls the greediness.
      - If size = 1: Pure Greedy (Deterministic).
      - If size = All: Pure Random Construction.

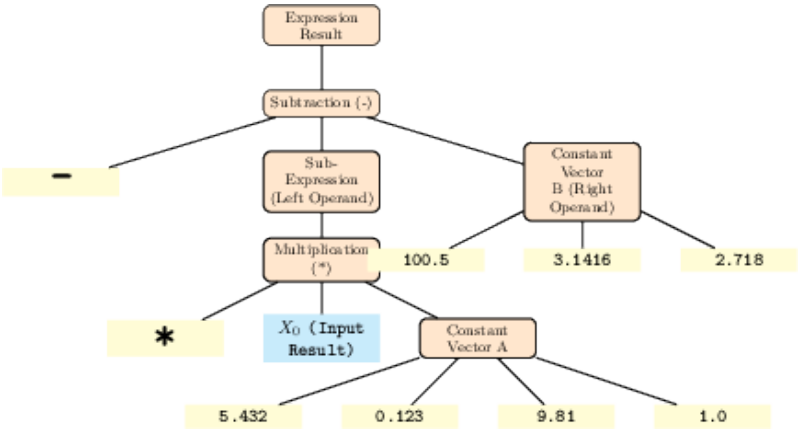
Evolutionary Algorithms

- Genetic Algorithm (GA)
  - Variant of stochastic beam search motivated by natural selection.

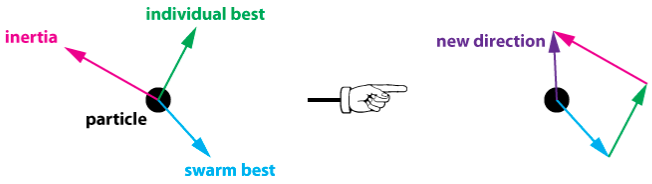
- **Components**:
  - ∗ **Population**: Set of individuals (states).
  - ∗ **Fitness Function**: Evaluation of the state.
- **The Cycle**:
  1. **Selection**: Choose parents based on fitness.
  2. **Crossover (Recombination)**: Combine parts of two parents to create offspring.
  3. **Mutation**: Randomly alter a bit in the offspring (prevents stagnation).
- **Schema Theory**: Genetic algorithms work best when solutions can be built by combining "blocks" of useful genes (schemas).
- **Scope**: Can solve **Multi-objective problems** (optimizing conflicting goals simultaneously, e.g., minimizing cost while maximizing strength).



- Genetic Programming (GP)
  - Evolves **programs** or **functions** rather than static data strings.
  - **Representation**: Individuals are **Trees**.
    - ∗ **Internal Nodes**: Functions/Operators (+, -, *, IF, SIN).
    - ∗ **Leaves (Terminals)**: Variables (x, y) or Constants (3.14).
  - **Crossover**: Swaps entire sub-trees between parent programs to create new logic.



- Particle Swarm Optimization (PSO)
  - Simulates social behavior (bird flocking or fish schooling) rather than evolution.
  - **Representation**: Individuals are **Particles** moving through the search space.
  - **Mechanism**: Each particle updates its velocity based on three vectors:
    1. **Inertia**: Keep moving in the current direction.
    2. **Cognitive Component (Personal Best)**: Pull towards the best position **this specific particle** has ever found.
    3. **Social Component (Global Best)**: Pull towards the best position found by **the entire swarm**.



`img/pso.gif`

V. Machine Learning as a Heuristic Bias

Where does the "Information" come from?

Traditional heuristics bias the search using the mathematical or logical properties of the **current problem itself** (e.g., "Choose the shortest edge").

- **What if the problem is too complex for human-engineered rules?**
- **The ML Shift:** We can use Machine Learning models to provide the bias, but the information source changes.
- Instead of deriving logic from the **current problem**, the model derives statistical intuition from **thousands of previously solved instances**.

What Can ML Learn to Bias?

Learned bias	Learns from	Decision it biases
Algorithm selection	Features of past instances	Which algorithm to run on a
Parameter configuration	Results of past runs	Parameter values (T_0, coolin
Surrogate models	Already evaluated solutions	Which candidates deserve a
Model-based sampling (EDAs)	Best solutions of the population	Where to sample the next so
Adaptive operator selection	Reward of recent moves	Which move / heuristic to a
Case-based warm start	Solved instances (k-NN)	The initial solution

Example: Learning Which Move to Apply

A multi-armed bandit (reinforcement learning) learns online which operator improves the solution most often, balancing **exploration** and **exploitation** as in the Secretary Problem.

```
import random

# Each operator improves the solution with an unknown
    probability.
success_rate = {"swap": 0.10, "insert": 0.20, "2-opt":
    0.35}
reward_sum = {op: 0.0 for op in success_rate}
uses = {op: 0 for op in success_rate}
epsilon = 0.1  # 10% exploration, 90% exploitation
```

```
for step in range(2_000):
    if random.random() < epsilon or step < len(
        success_rate):
         op = random.choice(list(success_rate))
                                              # explore
    else:
         op = max(success_rate, key=lambda o:
             reward_sum[o] / max(uses[o], 1))  # exploit
    improved = random.random() < success_rate[op]  #
        apply operator to the solution
    uses[op] += 1
    reward_sum[op] += improved

for op in success_rate:
    print(f"{op:7s} used {uses[op]:5d} times, learned
        success = {reward_sum[op] / max(uses[op], 1):.2
        f}")
```

```
swap    used    63 times, learned success = 0.13
insert  used   202 times, learned success = 0.17
2-opt   used  1735 times, learned success = 0.36
```

- Nobody told the algorithm that 2-opt is best: the bias was **learned from experience**.

VI. Summary

Comparison Summary

Method	Philosophy	Generality	Cost	Exa
Exact	Guarantees Global Optimum	Very High	Exponential	Bra
Greedy	Short-sighted, localized choices	Problem-specific	Low	Nea
Heuristics	Guided search, local optima traps	Problem-specific	Low/Med	2-op
Metaheuristics	Accepts bad moves to explore globally	General framework	High	GA,

Part 3: Algorithms that Use Approximation and Randomization

I. Approximation Algorithms

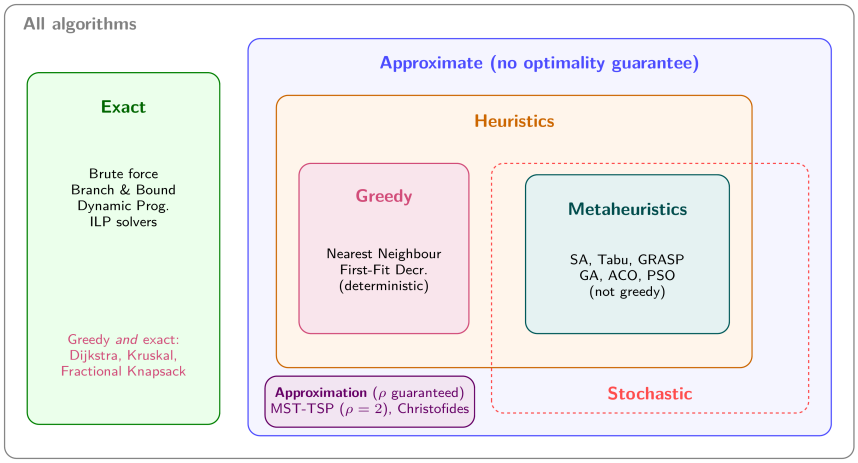
Approximation with a Guarantee

A heuristic **hopes** for a good solution. An **approximation algorithm proves** how far from the optimum it can be.

- **Approximation ratio ρ:** For every instance, in polynomial time, the algorithm returns a solution with $f(\text{ALG}) \leq \rho \cdot f(\text{OPT})$ (minimization).
- **Deterministic or randomized:** The guarantee comes from the analysis, not from randomness. The MST algorithm below is deterministic; randomized rounding of an LP is a randomized one.
- **Not always possible:** For the general TSP no constant ρ exists unless $\mathcal{P} = \mathcal{NP}$. We need extra structure in the problem.

Where Approximation Algorithms Fit

They are approximate (no proof of optimality) but, unlike heuristics, they come with a proven bound.

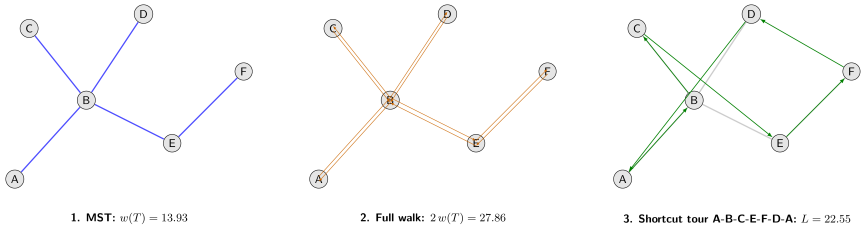


Metric TSP and the Triangle Inequality

- **Metric TSP:** Distances satisfy the **triangle inequality**: $d(a, c) \leq d(a, b) + d(b, c)$. Going directly is never longer than making a detour.
- Euclidean distances (cities on a map) are metric.
- **Consequence:** If a walk visits a city twice, we can **shortcut** it by jumping to the next unvisited city, and the walk never gets longer.

TSP Approximation Using a Minimum Spanning Tree

1. **Build a Minimum Spanning Tree (MST)** T of the cities (Prim or Kruskal).
2. **Walk around the tree** (depth-first), crossing each edge twice: cost $2\,w(T)$.
3. **Shortcut** repeated cities: keep only the first visit of each city (the **preorder** of the tree).



- The tour still crosses itself: a 2-opt step (Part 2) would improve it further.

Why It Is a 2-Approximation

- Remove one edge from the optimal tour: the result is a spanning tree, so $w(T) \leq f(\text{OPT})$.
- The full walk uses every MST edge twice: walk $= 2\,w(T) \leq 2\,f(\text{OPT})$.
- Shortcuts never increase the length (triangle inequality): $f(\text{ALG}) \leq$ walk $\leq 2\,f(\text{OPT})$.
- **Better bound:** Christofides' algorithm (MST + minimum perfect matching on the odd-degree vertices) reaches $\rho = 1.5$.

MST Approximation vs. Optimal Tour

```
import itertools
import math
import random

def mst_prim(points: list[tuple[float, float]]) ->
    dict[int, list[int]]:
    """Prim's algorithm from city 0; returns the tree
       as {node: children}."""
    best = {v: (math.dist(points[0], points[v]), 0)
       for v in range(1, len(points))}
    children = {v: [] for v in range(len(points))}
    while best:
        v = min(best, key=lambda u: best[u][0])
        children[best.pop(v)[1]].append(v)          #
           add the cheapest edge to the tree
        for u in best:
            d = math.dist(points[v], points[u])
            if d < best[u][0]:
                best[u] = (d, v)
    return children

def preorder(children: dict[int, list[int]], root: int
    = 0) -> list[int]:
    """Walk around the tree; visiting each city once =
        shortcutting repeated cities."""
    return [root] + [c for child in children[root] for
        c in preorder(children, child)]

def tour_length(points, tour) -> float:
    return sum(math.dist(points[tour[i]], points[tour
       [(i + 1) % len(tour)]]) for i in range(len(tour
       )))

random.seed(7)
points = [(random.uniform(0, 10), random.uniform(0,
   10)) for _ in range(9)]

approx = preorder(mst_prim(points))
```

```
optimal = min(((0,) + p for p in itertools.
    permutations(range(1, len(points)))),
                key=lambda t: tour_length(points, t))
                    # brute force: 8! = 40,320 tours

print(f"MST approximation: {approx}  L = {tour_length(
    points, approx):.2f}")
print(f"Optimal (brute):   {list(optimal)}  L = {
    tour_length(points, optimal):.2f}")
print(f"Ratio = {tour_length(points, approx) /
    tour_length(points, optimal):.3f}  (guaranteed <=
    2)")
```

```
MST approximation: [0, 7, 5, 4, 3, 2, 1, 6, 8]  L = 34.67
Optimal (brute):   [0, 1, 2, 8, 6, 3, 4, 7, 5]  L = 26.71
Ratio = 1.298  (guaranteed <= 2)
```

- In practice the ratio is usually far below the worst-case bound of 2.

II. Randomized Algorithms

Why Randomize?

A randomized algorithm uses random numbers to make decisions, so its behaviour is a **random variable** even on a fixed input.

- **No bad inputs:** Randomness breaks the link between an input and the worst case: an adversary cannot build a bad input for a random pivot.
- **Simplicity:** Randomized algorithms are often much shorter than their deterministic counterparts.
- **Sampling:** A random sample can estimate a quantity that is too expensive to compute exactly.

Monte Carlo vs. Las Vegas

Family	Running time	Answer
Monte Carlo	Fixed (bounded)	Correct with high probability, or an estimate
Las Vegas	Random (bounded in expectation)	Always correct

Monte Carlo Methods

Run for a fixed time and accept a small, **controlled** error.

- **Monte Carlo estimation:** Average N random samples. The error shrinks as $\mathcal{O}(1/\sqrt{N})$, whatever the dimension of the problem: 100Œ more samples buy one more correct digit.
- **Monte Carlo decision algorithms:** Answer yes/no with a bounded error probability.
  - **One-sided error:** one of the two answers is always right.
  - **Amplification:** repeat k independent times; an error of $\leq 1/4$ per run falls to $\leq (1/4)^k$.

Monte Carlo Estimation: Error vs. Samples

The π estimate from Part 1, with more and more samples:

```
import math
import random

random.seed(1)
for n in [10**2, 10**3, 10**4, 10**5, 10**6]:
    inside = sum(random.random()**2 + random.random()
        **2 <= 1.0 for _ in range(n))
    estimate = 4 * inside / n
    print(f"N␣=␣{n:>9,}␣␣␣pi␣~␣{estimate:.5f}␣␣␣error␣
        =␣{abs(estimate␣-␣math.pi):.5f}␣␣␣1/sqrt(N)␣=␣
        {1␣/␣math.sqrt(n):.5f}")
```

```
N =       100   pi ~ 3.20000   error = 0.05841   1/sqrt(N) = 0.10000
N =     1,000   pi ~ 3.12800   error = 0.01359   1/sqrt(N) = 0.03162
N =    10,000   pi ~ 3.14560   error = 0.00401   1/sqrt(N) = 0.01000
N =   100,000   pi ~ 3.13856   error = 0.00303   1/sqrt(N) = 0.00316
N = 1,000,000   pi ~ 3.14114   error = 0.00046   1/sqrt(N) = 0.00100
```

Monte Carlo Decision: MillerRabin Primality Test

```
import random

def is_probable_prime(n: int, k: int = 20) -> bool:
```

```
    """Miller–Rabin: 'False' is always right; 'True'
       is wrong with probability <= 4^-k."""
    if n < 4:
        return n in (2, 3)
    if n % 2 == 0:
        return False
    d, s = n - 1, 0
    while d % 2 == 0:
        d, s = d // 2, s + 1
    for _ in range(k):
        x = pow(random.randrange(2, n - 1), d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False           # found a witness: n
                is certainly composite
    return True                    # probably prime

for n in [561, 7919, 2**61 - 1, 2**61 + 1]:
    print(f"{n:>22,}: {'probably prime' if
       is_probable_prime(n) else 'composite'}")
```

```
                561: composite
              7,919: probably prime
2,305,843,009,213,693,951: probably prime
2,305,843,009,213,693,953: composite
```

- 561 is a Carmichael number: it fools the simpler Fermat test, but not MillerRabin.
- Used to generate the large primes of RSA keys, where a deterministic test would be far slower.

Las Vegas Methods

Never return a wrong answer; only the running time is random.

- **Expected running time:** An average over the algorithm's own random choices, **for every input**.
- **Randomized QuickSort:** A random pivot gives expected $\mathcal{O}(n \log n)$ on every input. The $\mathcal{O}(n^2)$ case still exists but becomes extremely unlikely.
- **Other examples:** QuickSelect (median in expected $\mathcal{O}(n)$), random restarts for the n-Queens problem, universal hashing.
- **Converting:** Stop a Las Vegas algorithm at a time limit and it becomes Monte Carlo. A Monte Carlo algorithm whose answer can be **verified** becomes Las Vegas: repeat it until the check passes.

Las Vegas Example: Randomized QuickSort

```
import math
import random
import sys

sys.setrecursionlimit(10_000)

def quicksort(a: list[int], pick_pivot, counter: list[
    int]) -> list[int]:
    if len(a) <= 1:
        return a
    p = pick_pivot(a)
    counter[0] += len(a) - 1                          #
        comparisons against the pivot
    left = [x for x in a if x < p]
    right = [x for x in a if x > p]
    return quicksort(left, pick_pivot, counter) + [p]
       + quicksort(right, pick_pivot, counter)

n = 2_000
data = list(range(n))                                 #
    already sorted: worst case for a fixed pivot
for name, pick in {"First element": lambda a: a[0], "
    Random pivot": random.choice}.items():
    counter = [0]
```

```
    assert quicksort(data, pick, counter) == data     # Las Vegas: the answer is always correct
    print(f"{name:14s} comparisons = {counter[0]:>9,}")
print(f"{'n log2 n':14s}                 = {round(n * math.log2(n)):>9,}")
```

```
First element  comparisons = 1,999,000
Random pivot   comparisons =    26,327
n log2 n                   =    21,932
```

- On sorted input the fixed pivot does about $n^2/2$ comparisons; the random pivot stays close to $n \log_2 n$. Both return the same sorted list.

III. Summary

Comparison Summary

Technique	Gives up	Keeps	Example
Approximation	Optimality	A proven bound ρ	MST-TSP ($\rho = 2$)
Monte Carlo	Certainty	Fixed running time	MillerRabin, MC integration
Las Vegas	Fixed running time	Always-correct answers	Randomized QuickSort
Heuristics (Part 2)	Any guarantee	Speed	2-opt, Nearest Neighbour

Part 4: Algorithms that Trade Space for Time

I. The SpaceTime Trade-off

Spending Memory to Save Time

Store results once, look them up many times.

- **Lookup tables:** Precompute values (trigonometric tables, CRC tables, the parity of every byte).
- **Caches and indexes:** Hash tables, database indexes and web caches answer in $\mathcal{O}(1)$ or $\mathcal{O}(\log n)$ instead of a full scan.
- **The other direction:** Compression and recomputation save memory at the cost of time.

- **Another multi-objective choice:** As with quality vs. resources in Part 2, time and memory compete, and the right point depends on the hardware and the problem size.

II. Memoization

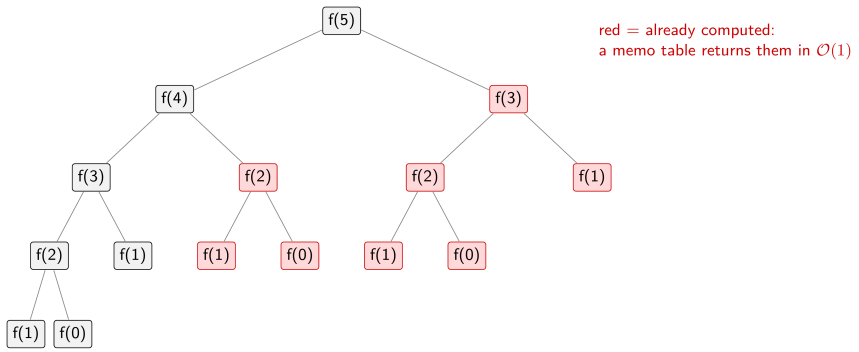
Memoization: Remember Every Answer

Store the result of each function call in a table; when the same arguments come back, return the stored result.

- **Only for pure functions:** The same arguments must always give the same result (Part 1, Functional Programming). A function with side effects cannot be memoized safely.
- **Worth it when calls repeat:** It only pays off if the same sub-calls appear many times (**overlapping subproblems**).
- **In Python:** `@functools.cache`, or `@functools.lru_cache(maxsize=...)` to bound the memory it uses.

Repeated Calls in Naive Recursion

$F(n) = F(n-1)+F(n-2)$: the naive recursion recomputes the same values again and again.



Memoized Fibonacci

```
calls = {"naive": 0, "memoized": 0}

def fib_naive(n: int) -> int:
```

```
    calls["naive"] += 1
    return n if n < 2 else fib_naive(n - 1) +
        fib_naive(n - 2)

memo: dict[int, int] = {}            # the extra space
    we pay

def fib_memoized(n: int) -> int:
    calls["memoized"] += 1
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib_memoized(n - 1) + fib_memoized(n
            - 2)
    return memo[n]

n = 30
print(f"fib({n}) = {fib_naive(n)} with {calls['naive
    ']:,} calls (naive)")
print(f"fib({n}) = {fib_memoized(n)} with {calls['
    memoized']:,} calls and {len(memo)} stored values (
    memoized)")
```

```
fib(30) = 832040 with 2,692,537 calls (naive)
fib(30) = 832040 with 59 calls and 29 stored values (memoized)
```

- **Naive:** $\mathcal{O}(\varphi^n) \approx \mathcal{O}(1.618^n)$ time, $\mathcal{O}(n)$ stack.
- **Memoized:** $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ extra memory for the table.

III. Dynamic Programming

From Memoization to Dynamic Programming

Dynamic Programming (DP) solves a problem by combining the stored solutions of its subproblems.

- **Principle of Optimality:** The optimal solution is built from optimal solutions of subproblems (Bellman, seen with greedy algorithms in Part 2).

- **Overlapping subproblems:** The same subproblems appear many times, so storing them pays off.
- **DP vs. Greedy:** Greedy commits to one choice per step. DP evaluates **every** choice for every subproblem and keeps the best one, which is why it is exact.
- **Two styles:**
  - **Top-down:** recursion + memoization (only the needed subproblems are solved).
  - **Bottom-up (tabulation):** fill a table from the smallest subproblems up (no recursion stack).

DP for the 0/1 Knapsack

$K[i][c]$ = best value using items $1..i$ with capacity c:

$$K[i][c] = \max\Big(\underbrace{K[i-1][c]}_{\text{skip item } i},\ \underbrace{v_i + K[i-1][c-w_i]}_{\text{take item } i \text{ (if } w_i \le c\text{)}}\Big)$$

```
def knapsack(values: list[int], weights: list[int], W:
    int):
    n = len(values)
    K = [[0] * (W + 1) for _ in range(n + 1)]    # K[i
        ][c]: best value with items 1..i and capacity c
    for i in range(1, n + 1):
        v, w = values[i - 1], weights[i - 1]
        for c in range(W + 1):
            K[i][c] = K[i - 1][c]
                                                    #
                skip item i
            if w <= c:
                K[i][c] = max(K[i][c], v + K[i - 1][c
                    - w])          # take item i
    chosen, c = [], W                               # walk
        back through the table
    for i in range(n, 0, -1):
        if K[i][c] != K[i - 1][c]:
            chosen.append(i - 1)
            c -= weights[i - 1]
```

```
    return K, K[n][W], chosen[::-1]

items, values, weights = ["A", "B", "C"], [60, 100,
    120], [10, 20, 30]
K, best, chosen = knapsack(values, weights, 50)
print("capacity:", *[f"{c:4d}" for c in range(0, 51,
    10)])
for i, row in enumerate(K):
    print(f"{'-' if i == 0 else '+' + items[i - 1]:>8s
        }:", *[f"{row[c]:4d}" for c in range(0, 51, 10)
        ])
print(f"Best value = {best} with items {[items[i] for
    i in chosen]}  (greedy found 160)")
```

```
capacity:    0   10   20   30   40   50
      -:     0    0    0    0    0    0
     +A:     0   60   60   60   60   60
     +B:     0   60  100  160  160  160
     +C:     0   60  100  160  180  220
Best value = 220 with items ['B', 'C']  (greedy found 160)
```

Knapsack DP: Cost and Trade-offs

- **Time and space:** $\mathcal{O}(nW)$ for the full table.
- **Pseudo-polynomial:** Polynomial in the **value** of W, but exponential in the number of bits needed to write W. This does not contradict NP-hardness (Part 2).
- **Trading back:** Keeping only the previous row reduces space to $\mathcal{O}(W)$, but then the chosen items cannot be recovered from the table.
- **Other classic DPs:** Shortest paths (BellmanFord, FloydWarshall), edit distance, longest common subsequence, and HeldKarp for the exact TSP in $\mathcal{O}(n^2 2^n)$: far better than $\mathcal{O}(n!)$, but still exponential.

IV. Summary

Comparison Summary

Approach	Time	Extra space	Example
Naive recursion	$\mathcal{O}(\varphi^n)$	$\mathcal{O}(n)$ stack	Fibonacci
Memoization	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Fibonacci
DP table	$\mathcal{O}(nW)$	$\mathcal{O}(nW)$	0/1 Knapsack
DP, one row	$\mathcal{O}(nW)$	$\mathcal{O}(W)$	0/1 Knapsack (value only)
HeldKarp	$\mathcal{O}(n^2 2^n)$	$\mathcal{O}(n2^n)$	Exact TSP

Exercises and References

Exercises

1. **Parallel Speedup:** Run `estimate_pi` with 1, 2, 4 and 8 processes and plot the speedup. Why is it not linear?

2. **Algorithmic Analysis:** Modify the greedy knapsack algorithm to prioritize items with the lowest weight first. Show a small instance where this performs worse than the value/weight ratio method.

3. **Local Search:** Implement 2-opt for the TSP and count how many improvements are made before reaching the local optimum. Does starting from a Nearest Neighbour tour require fewer improvements than a random tour?

4. **Parameter Tuning:** Implement Simulated Annealing from the pseudocode and change the cooling schedule. How does it affect the final solution quality and execution time?

5. **Approximation Ratio:** Run the MST approximation on 100 random instances of 9 cities and plot the ratio against the optimal tour. How close does it get to 2? What happens if you apply 2-opt after it?

6. **Las Vegas vs. Monte Carlo:** Solve the 8-Queens problem by placing queens at random and restarting on conflict. Measure the distribution of the number of restarts, then turn it into a Monte Carlo algorithm with a restart limit.

7. **Space vs. Time:** Rewrite the knapsack DP to use a single row ($\mathcal{O}(W)$ space). Then solve it top-down with `@functools.cache` and count how many subproblems are actually computed.

Bibliography

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). **Introduction to Algorithms**. MIT press.
- Talbi, E. G. (2009). **Metaheuristics: From Design to Implementation**. John Wiley & Sons.
- Vazirani, V. V. (2001). **Approximation Algorithms**. Springer.
- Motwani, R., & Raghavan, P. (1995). **Randomized Algorithms**. Cambridge University Press.
- Russell, S., & Norvig, P. (2021). **Artificial Intelligence: A Modern Approach** (4th ed.). Pearson. (Chapter 4: Local Search)
- Burke, E. K., Gendreau, M., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., & Qu, R. (2013). **Hyper-heuristics: A survey of the state of the art**. Journal of the Operational Research Society, 64(12), 1695-1724.
- Rice, J. R. (1976). **The algorithm selection problem**. Advances in Computers, 15, 65-118.
- López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L. P., Birattari, M., & Stützle, T. (2016). **The irace package: Iterated racing for automatic algorithm configuration**. Operations Research Perspectives, 3, 43-58.
- Wolpert, D. H., & Macready, W. G. (1997). **No free lunch theorems for optimization**. IEEE transactions on evolutionary computation, 1(1), 67-82.